



**Simon says:**

## Keep It Simple and Complete

By Simon Brown, independent consultant and author of *Software Architecture for Developers* [Bro16]

Having run diagramming workshops for more than 10,000 people around the world, I can say with some confidence that software developers struggle to communicate software architecture using diagrams. Many of the diagrams I see focus on the decomposition of software into logical building blocks, although they usually do this at the expense of information about technology. After all, we've been taught to keep the "logical view" of a software system separate from the "development view." Unfortunately, this tends to result in a collection of diagrams that make little sense when viewed individually, with neither diagram providing an accurate reflection of the code.

One solution is to merge the logical and development views of a software system, treating the combined diagrams as a collection of zoomable maps on top of a code base. This is the concept at the core of my *C4 model*—a collection of diagrams that show varying levels of abstraction. A *System Context diagram* shows your software system and how it fits into the environment around it, in terms of users and other software systems. A *Container diagram* zooms into your system boundary to show how it's made up of containers (applications and data stores). A *Component diagram* zooms in to a particular container to show the components inside it. Optionally, a UML class diagram zooms into a particular component to show the code-level elements it's built from.

Although UML is still useful, I prefer a simple *boxes and lines* notation to describe software architecture. To avoid confusion, my advice is to keep the notation as simple and self-describing as possible, adding a key/legend where necessary. Finally, adding more text to boxes is a great way to add information about responsibilities, providing a nice, at-a-glance view and removing ambiguity from what is usually just a collection of named boxes.

We want the patterns to be visible in the diagrams we draw. We can accomplish this in a variety of ways. Choose names that reflect the pattern. Create views that highlight the patterns we've selected. Arrange the elements so you can see the pattern. Above all, use patterns as part of your vocabulary for sharing designs.

## Strive for Consistency and Simplicity

Every drop of ink in our diagrams means something. Color, shape, orientation, font choice, and position convey meaning. Avoid superfluous details to help readers focus on the ideas you want to share. Choose different colors or shapes to highlight different ideas, not simply to make the diagram look pretty.

When sharing ideas, consistency is king. If an idea from the conceptual meta-model is present in two different diagrams, use the same shapes and colors to represent it. Readers will find meaning behind every change in color and font, even if you didn't mean anything by it.

Likewise, too many details will overwhelm and confuse readers. Sometimes we'll need multiple arrow heads to express an idea. Other times, too many arrows obfuscate meaning. Work to create the simplest diagrams as possible that still provide meaning.

## Provide Descriptive Prose

The stories that accompany the picture are the most interesting part of a view. The story explains how elements in the view come together to promote or inhibit quality attributes and why we designed the system the way we did. Sometimes the diagram is the least interesting part of a view. All the action is in the story!

We'll use *descriptive prose* to tell the stories about our architecture. Descriptive prose can be a simple table, paragraphs of text, bullet points, or even a verbal narrative. Think of diagrams as visual aides for stories about the architecture: where the architecture came from, how it works, where it's going.

In [Chapter 11, \*Describe the Architecture\*, on page 143](#) you will learn how to share the stories that accompany views. Diagrams are a vehicle for communicating design decisions and context, but diagrams can't speak for themselves. It's up to you to provide the story.



## Get Your Hands Dirty: Critique Some Diagrams

Find some diagrams from a recent software system you helped build. Knowing what you know now about how to visualize the architecture, what would you change about these diagrams? If this project is still in progress, try to improve the view and share it with your team.

Here are some things to think about:

- What does the diagram help you to reason about?
- What are the essential patterns in the diagram? Are there hidden patterns?
- What is the underlying meta-model? Can it be discerned from the diagram alone?
- Is the diagram complete? Could it be simplified and still be effective?



**Joe asks:**

## **Should I Use an Architecture Description Language?**

We're not going to cover formal Architecture Description Languages (ADLs) in this book, but you should be aware that such a thing exists. Most architects use simple drawing tools such as PowerPoint, Visio, and Graphviz for day-to-day work. Some people even use pencil, paper, and a phone camera to great effect.

Simple diagramming tools are easy to use and produce shareable diagrams, but pictures don't easily facilitate in-depth analysis. ADLs solve this problem by restricting and enforcing the vocabulary used to specify models. ADLs are usually implemented in tools, which can run automated checks against the models. Some ADLs can even generate or reverse-engineer code.

ADLs sound awesome but in practice don't always make life easier. The ADL you select will limit expressiveness of the design. Software tools that support ADLs will often save models in proprietary formats. In my experience, most tools are immature and have a steep learning curve.

My advice is to use an ADL only if you really, really want to use one. You can peruse an up-to-date list of ADLs at <http://www.di.univaq.it/malavolta/al/>.

## **Project Lionheart: The Story So Far...**

Software development is in full swing and progressing nicely. Every day we learn something new about the problem we're solving. The architecture is becoming more mature with every detail. The team draws pictures regularly to bounce ideas off each other and work through alternatives.

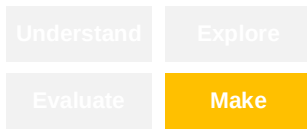
Teammates frequently sketch the same diagrams during design discussions. You take a picture of some whiteboard sketches with your phone and add the pictures to your source code repository along with with a brief write-up. You also start to build out element responsibility views so it's easier to explain how everything works together. So far these views have been most useful when we add new elements to the architecture.

Up to this point in the project we've only shared design decisions informally. And we've had a few misses because of this. You decide it's probably time for us to improve our documentation and rely less on tribal knowledge.

## Next Up

When trying to share an abstract idea, just start drawing. Do not hesitate. As you sketch, the abstract becomes tangible and new insights will emerge. Teammates will join you as you reason through the complex. Use these visual aides to tell stories about how your design decisions promote quality attributes. As the stories and diagrams coalesce, a view of the software system will emerge. Every view is a window into the architecture.

In the next chapter, you're going to learn more about telling stories with your diagrams by creating architecture descriptions. Architecture descriptions start with a picture but also include design decisions, history, and the rationale for why you designed the system the way you did.



## CHAPTER 11

# Describe the Architecture

---

Software architecture documentation has a reputation for being notoriously awful. It takes time away from writing code. It always seems to be out of date. It's usually written in some proprietary binary file format that you can't edit. And on top of all that, nobody reads it anyway! It's no wonder some people call software architecture descriptions SAD!

Bad architecture descriptions make us sad. Great architecture descriptions give our team clarity of vision. Great architecture descriptions are a planning asset, a communication aide, and a collaboration tool. They improve the quality of the software we build by helping our design decisions reach everyone.

In this chapter, you'll learn how to create amazing software architecture descriptions that people love. Why will people love them? Because you'll give your audience the exact information they need in a humane and easy-to-learn format. Love is a strong word to apply to software architecture descriptions. I mean it. Creating an amazing software architecture description is easier than you may realize.

## Tell the Whole Story

In [Build Models into the Code, on page 107](#) you learned that there will always be a gap between the models in the architecture and the code we write. We can close the model-code gap somewhat, but we can't express every architectural design decisions in code. Nevermind that few stakeholders can read code and no code exists at the beginning of a system's life when design planning is most needed.

We need architecture descriptions for several important reasons.

**Get organized.** Building software is as much about working with people as it is working with technology. Architecture descriptions show how everything comes together. Whether we are assigning responsibilities to teams or letting people self-organize, everyone needs to know how components in the system work together so they can align themselves accordingly.

**Establish the lingua franca between technical and business stakeholders.**

Every stakeholder has a right to understand the architecture. While models establish the vocabulary of design, the architecture description translates those models into ideas different stakeholders can understand. One of the most important jobs of an architecture description is to show how the business goals and quality attributes are addressed in the architectural design decisions.

**Put a spotlight on quality attributes.** Quality attributes are often out of sight and out of mind. The architecture is the one place we treat quality attributes as first-class citizens. Keep quality attributes at the front of everyone's mind by making the architecture a real thing that people can see, read about, and talk about.

**Clarify thinking.** It's easy to believe you have everything figured out if nothing is written down. When you put pen to paper, you'll realize how mushy the ideas in your head really are. Writing an architecture description forces us to confront our knowledge and come to terms with what we know, what we think we know, and what we don't know.

**Create something you can evaluate.** We can't reason about things we can't see, touch, or share. We also can't afford to wait for every design decision to be realized in code before critically evaluating it. Architecture descriptions give us a way to analyze our design decisions while it's easy to make changes. Spending the afternoon explaining a dumb idea to someone is much better than spending a month implementing it.

**Show it off.** Software architecture is cool! You poured your heart and soul into designing a beautiful software system. The whole world should appreciate it! Architecture descriptions are the best way to brag about the software systems you've built. A good architecture description projects confidence for customers and upper management. It shows leadership by clearly articulating a purpose, plan, and vision.

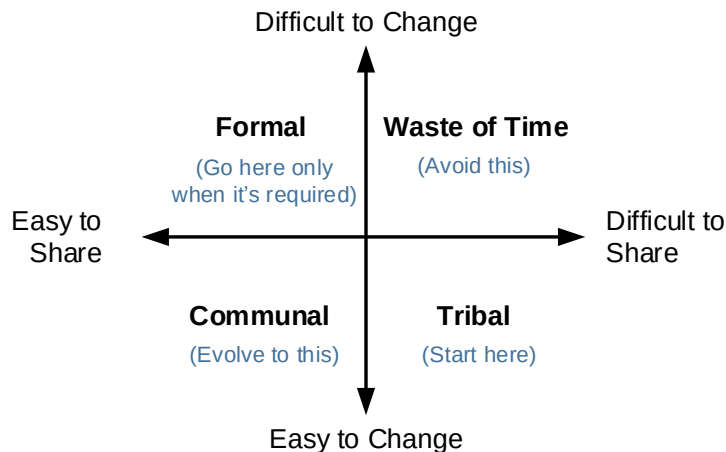
Architecture descriptions are supremely valuable. The trick to getting the most bang for the buck is to choose the appropriate description methods for your current project and team context.

## Match the Description Method to the Situation

There is no one-size-fits-all approach to creating software architecture descriptions. Less mature systems might change their architecture several times in a single conversation. A co-located team working on a smaller system can get by with whiteboards and storytelling. A system built for a regulated industry may be legally required to document design decisions in a specific way.

There are two questions we need to answer to decide how to document our architecture. How likely are our design decisions to change? And how far must we share our design decisions? Depending on how you answer these questions, you'll arrive at one of four types of description methods: tribal, communal, formal, or wasteful.

Which architecture description approach should I use?



## Create an Oral History with Tribal Methods

Tribal description methods rely heavily on oral tradition and cultural artifacts. Storytelling, metaphors, and informal sketching are all examples of tribal architecture descriptions. Always start here. Tribal descriptions are easy to change, which is perfect for the rapid design churn young architectures face.

While tribal methods are easy to create and change, they are also difficult to share. A system metaphor, described in [Activity 29, \*System Metaphor\*, on page 281](#), might work well when your whole team is within earshot of your conversation, but oral histories are only alive when someone can tell the story.<sup>1</sup> Constantly telling stories can be exhausting, even on small teams of only half a dozen people.

## Reach Further with Communal Methods

Not all aspects of an architecture evolve at the same rate or require the same kinds of description. As a rule of thumb, if you find yourself telling the same story to more than a few people, then it's time to evolve the style of architecture description to increase reach. Enter communal methods.

Communal architecture description methods are shared by the community, not just individuals of the tribe. Architecture haiku ([Activity 21, \*Architecture Haiku\*, on page 263](#)), architecturally evident coding styles ([described on page 107](#)), and architecture decision records ([Activity 20, \*Architecture Decision Records\*, on page 260](#)) are all examples of communal description methods. Communal descriptions are still easy to change, but they are also easy to share compared to tribal methods.

Most teams will evolve from tribal descriptions to communal descriptions naturally as the architecture matures and the rate of change decreases. Communal description methods are good enough for many teams. In some situations, we want something more permanent. We can get this permanence with formal architecture descriptions.

## Invest in Formal Descriptions Only When Required

Formal description methods include traditional architecture description documents and formal models. These tend to be larger documents and require more effort to create. Formal models (the sort that defines the architecture with a mathematical model) require a higher degree of accuracy and precision.

High-risk systems or architecture decisions requiring a high degree of coordination are good candidates for formal description methods. Depending on your industry, you may be required to create a formal document. Even then, it pays to start with tribal methods and progress to communal methods before building formal descriptions.

---

1. Michael Keeling. *Creating an Architecture Oral History: Minimalist Techniques for Describing Systems*. SATURN 2012. <http://resources.sei.cmu.edu/library/asset-view.cfm?assetID=20330>



Instead of starting with a traditional document, start with whiteboard sketches and a system metaphor. Record ADRs one at a time as you make decisions. Once you've made some decisions, bring everything together as an architecture haiku and continue refactoring code so that it reflects the models in your design. After the architecture starts to coalesce, if desired (or when a stakeholder asks for it), create a traditional document.

### Create a Traditional Software Architecture Description

The *traditional software architecture description*, or SAD, is a classic design document that every architect should know how to write. While these documents are time-consuming to create, they are worth their weight in gold. This does not mean you should make the document as large as possible! What I mean is that a traditional SAD is well worth the effort.

Most stakeholders—developers included—have probably never seen the architecture as a whole. The SAD is the one artifact that brings everything together to tell the whole story.

Traditional architecture documents might run 10 or 20 pages depending on the template used. I've written only a few SADs that have run 70 pages or more. Keep in mind these page counts include all the pomp and circumstance that goes along with a formal document. Even not counting the fluff, a traditional SAD is an investment.

Start a traditional architecture description by building or finding a template. You'll find many templates available online.<sup>2</sup> Your organization may even have a template. I strongly recommend the Software Engineering Institute's *Views and Beyond*<sup>3</sup> and the ISO/IEC/IEEE 42010 standard templates.<sup>4</sup>

All traditional architecture descriptions include the same basic parts:

**Introductions and preamble information** This includes the title page, update notes, signature page, table of contents, a list of figures, licensing and legal boilerplate, and other information required of a formal document. The table of contents and figure lists help readers navigate the document more easily. The rest is meant to convey the importance of the information held within the document. Some stakeholders find these preambles impressive. Remember, you may be required by your organization to include some of this information.

2. <http://www.iso-architecture.org/42010/applications.html>

3. <http://www.sei.cmu.edu/architecture/tools/document/viewsandbeyond.cfm>

4. <http://www.iso-architecture.org/42010/templates/>

**Overview and introduction to the SAD** Briefly describes the purpose of the document as well as the methodology used to organize and create it. Your SAD could be the first time some stakeholders have read an architecture description. Take this opportunity to educate them just-in-time so they can appreciate the architecture designed for them.

**Summary of stakeholders, business goals, and architecturally significant requirements** Since all decisions in our architecture flow from stakeholders' concerns, list them before describing the design. I like to summarize key constraints and quality attributes here as well. If you've created an ASR Workbook ([introduced on page 60](#)), then add a reference to it. Strive to keep the architecture description DRY (Don't Repeat Yourself), just like your code.

**Context view** Provides an overview of where the software system fits in the world. See [Activity 22, Context Diagram, on page 265](#) for details.

**Relevant views** As we discussed in [Show the Architecture from Different Views, on page 129](#), architecture is too big and complex to show in one diagram. We need to create multiple views of the architecture to explain how it satisfies quality attributes and other requirements. A list of views is not very consumable, so to help our readers we'll organize views around a related set of stakeholder concerns. Each *viewpoint* shows views needed to reason about something a stakeholder cares about, such as a set of related quality attribute scenarios. You'll learn more about using viewpoints in [Organize Views around Stakeholders' Concerns, on page 152](#).

**Risks, open questions, future work** Include a section for known risks and open questions. The purpose of these sections is to shine a light on the land mines you already know about so downstream designers can hopefully avoid them.

**Appendices** At a minimum include a term glossary and list of acronyms with expansions. I recommend you include a quality attribute taxonomy as an appendix as well. Some formal documents will also include change procedures and change request templates.

Creating a SAD can be exhausting. Work as a team to complete the document. Designate one person as the *Master of the SAD*. The Master of the SAD creates the template and decides who will write each section of the document. The Master of the SAD is also responsible for making sure the document is complete and has a consistent style.

## Avoid Wasting Time

For the sake of completeness, there is a quadrant on our grid made up of difficult-to-change, difficult-to-share description methods. If you find yourself here, it's time to try something different. Let's look at two examples of waste-of-time description methods.

One example of a possible waste of time is the *slideware architecture description*. Presentations are a powerful tool for architects. The problem with slides is that they rarely stand on their own and can be difficult to change. Someone must present them to make sense. Someone spent hours getting all the transitions and layered boxes and connecting arrows to look just right. After so much work, people hesitate to change their beautifully laid-out diagrams, even when the world has changed. Compared to tribal methods, slides are etched in stone.

The best way to avoid wasting time is by creating great architecture descriptions. All great descriptions, no matter what method you use, have four traits:

1. They are custom built with the audience in mind.
2. They show multiple views of the architecture.
3. They clearly define the elements and their responsibilities.
4. They explain the rationale for design decisions.

In the next sections, you'll learn what these traits mean and how to put them into practice to create fantastic architecture descriptions.

## Respect Your Audience

As you learned in [\*The Four Principles of Design Thinking\*, on page 15](#), all design is social. Who will use your architecture description? What do they need to get out of it? How can you best provide the information they need most? When empathizing with our stakeholders, we identified and recorded their key concerns. We've come up with a plan for addressing their concerns in the architecture and now our job is to share that information.

When you know your audience, you'll be able to create an architecture description that gives them exactly what they need. The better we do this, the more likely people are to read the architecture description, which in turn further amplifies the impact of our design decisions.



**George says:**

## Tell a Story at Many Levels

By George Fairbanks, software engineer at Google and author of *Just Enough Software Architecture: A Risk-Driven Approach* [Fai10]

We've all dropped into a project and struggled to understand the code. Perhaps you are browsing the repository of an open source project and find just one folder with hundreds of source files. It takes a lot of effort for you to infer how things work and you probably make mistakes.

It doesn't have to be that way. You can organize your code as a *story at many levels* so that it makes sense and tells a story as you zoom in or out. Consider what you ate yesterday evening. You recognize it as dinner, and zooming in you also recognize courses, then dishes, then ingredients.

The levels help you think clearly. If you're wondering how long dinner will last, thinking about the number of courses is helpful and thinking about allergies is best done by ingredients. But the reverse doesn't help at all.

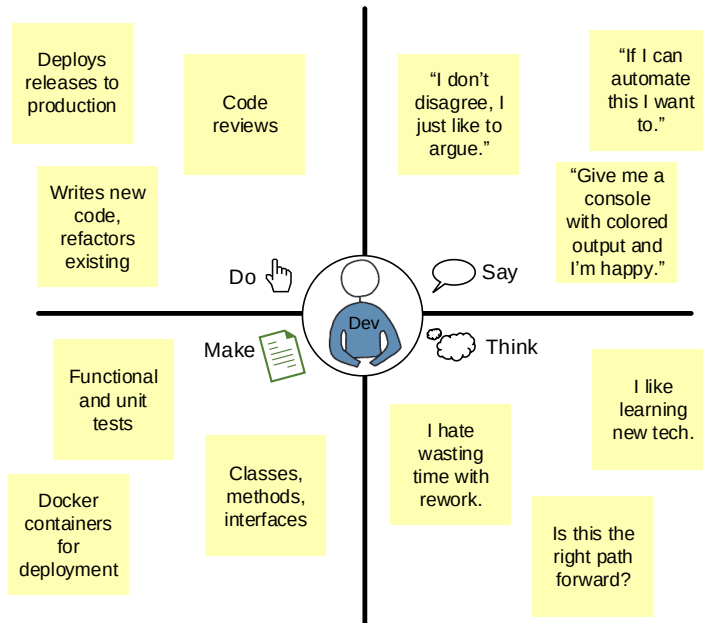
People have been thinking about dinner for a lot longer than software, so it's already baked into our language. You will have to invent your story and levels, though you can lean on developers who have come before you. Architecture patterns give you general names to use like *connectors* and *layers*, and specific ones like *reduce stage* and *broker*.

Your story at many levels won't come for free, so you will spend time gardening, making minor refactorings as you go so that the story stays clear. Today you might have just three connectors, so they all go in the same folder, but some well-timed gardening should catch it before it grows to dozens. As with most things, a stitch in time saves nine, plus it makes you look like you knew what you were doing the whole time.

It's easy to focus your attention on the source code only, but the system's runtime deserves similar attention, as does how it is allocated to hardware or containers. If you want to think about it clearly, structure it as a story at many levels.

Think about your stakeholders and what they value. What are their roles and responsibilities on the project? How do they like to process information? How will this person use the information you give them? Sometimes it helps to create an empathy map of your audience to help you get started (see [Activity 2, Empathy Map, on page 195](#)). There's a sample empathy [map on page 151](#) for a developer on my team.

By studying the example empathy map, we can see that this person needs a thorough document with a clear rationale since he likes to argue both sides of any decision. We can also see that this person is interested in properties related to deployment and wants technical details.



Now that we know what kind of information our audience wants and how they might consume it, let's talk about how we can make the most understandable architecture description for them.

## Focus on Understandability

Communication is king when it comes to architecture descriptions. Your audience wants to understand you. Speak the language of the domain in which you're working to make your architecture description understandable. If your stakeholders talk about *material master numbers*, use their terms instead of introducing new words like *product IDs*.

How you choose to describe complex, abstract ideas is also important. Use plain speak and avoid jargon. Briefly define architecture concepts to ensure your reader has required background knowledge. Unless it is completely wrong, favor design terminology stakeholders already use. For example, if stakeholders prefer to discuss *nonfunctional requirements* then use that term instead of *quality attributes*. Err on the side of effective communication.

Understandability also extends to notations. Not everyone will know every design notation. This advice goes double for the Unified Modeling Language (UML). There are several flavors of the UML, and though it can express architectural ideas well, it's not always direct or obvious. You may know

exactly how to use all of UML’s 16 or so diagrams, but not everyone in your audience has studied UML as intensely. As you learned in [Draw Fantastic Diagrams, on page 136](#), always define your notation and the meta-model behind it by including a legend on all diagrams.

Finally, organize your description so it’s easy to consume. Use a standard template for written documents and make it look good. Fix the alignment so it’s easy to read. Use text formatting to your advantage. Think about how the information looks in print, digital, and presentation formats. A great-looking document tells readers the content is trustworthy and was created by a professional.

| Do this...                                                    | Avoid this...                                                    |
|---------------------------------------------------------------|------------------------------------------------------------------|
| Do define architectural concepts the first time you use them. | Don’t unnecessarily introduce new concepts.                      |
| Do speak the language of the problem domain.                  | Don’t assume everyone intuitively understands diagram notations. |
| Do include a legend on diagrams.                              | Don’t use jargon.                                                |
| Do use a common template if one exists.                       |                                                                  |

You’ve learned a few techniques for documenting design decisions. You’ve also learned how to think about your audience to design an architecture description that is right for them. Next, we’ll combine these two ideas together by organizing the architecture description relative to stakeholders’ concerns.

## Organize Views around Stakeholders’ Concerns

Different people want to know different things about the software system you are building. Developers on your team will want to know about code organization, deployment, and component interaction. Testers on your team will want to know about the interfaces and communication protocols. The product owner on your team wants to know about technical dependencies and get a sense of the overall progress. New teammates might be overwhelmed by the existing documentation and could benefit with some help getting started. At a minimum, architecture descriptions should describe design decisions, design rationale, and structures in the design.

How we organize this information is important too. The Human rule of design applies equally to how we share details *about* the design just as much as it applies to the design itself.

When we organize views of the architecture and the various design documentation that goes with it with our stakeholders in mind, then it’s far easier for

others to understand the architecture. Doing this requires that we think about what stakeholders want to know. We can then create a view of the architecture unique to that set of related stakeholder concerns. Designing usable documents is how we get people to love our architecture descriptions.

### Establish the Viewpoints

A *viewpoint* defines an approach for describing the architecture from the perspective of a related set of stakeholder concerns. Viewpoints define not only what views you should show but also who the views are for, as well as the notations, vocabulary, and rules to use when creating it. They are a part of the ISO/IEC/IEEE 42010:2011 standard defined in [ISO/IEC/IEEE 42010:2011 Systems and software engineering – Architecture description \[Int11\]](#).

Viewpoints were designed for use with traditional architecture descriptions, but the general principles apply to any architecture description approach we've discussed. Let's look at an example.

We've identified several components in Project Lionheart. Eventually, we'll need to deploy these components somewhere. The development team needs to know where to deploy them, and the city IT department will need to be told what systems to monitor. It sounds like we need a deployment viewpoint. Here's one possible view from that viewpoint:

| Component                      | Deployed to                                                                                          |
|--------------------------------|------------------------------------------------------------------------------------------------------|
| Web UI                         | Runs in user's browser, served by Display Business (accessed through load balancer)                  |
| Display Business               | Tomcat on a Linux VM, hosted by cloud provider                                                       |
| Favorites Service              | Tomcat on a Linux VM, hosted by cloud provider, VM is independently deployable from Display Business |
| Search Service                 | Same VM as Favorites service                                                                         |
| Search Index                   | Cloud-provided Solr service                                                                          |
| Alerting Process               | Process initiated by cloud scheduler, independent container (depends on cloud provider)              |
| Crawler Process                | Local server maintained by City IT                                                                   |
| Contracts and User Metadata DB | Cloud-hosted Postgres                                                                                |

Additional views for the deployment viewpoint will flesh out dependencies among components, platform or third-party software requirements (for

example, a particular Linux version), areas of risk, costs, or network topology. The viewpoint itself can consist of both graphical and textual views.

In practice, the information in this example view can be captured in a few ADRs or summarized in an architecture haiku. For the Project Lionheart team, it might be enough to capture the tribal knowledge in the deployment scripts themselves. Add some comments to the scripts to capture design rationale. While this is OK for the development team, a summary table like the one shown will help with the project hand-off to City IT. Always consider the audience when deciding how to document the architecture.

## Create Custom Viewpoints

There are several established viewpoint sets you may use to guide your architecture description practices.<sup>5</sup> I can personally recommend the [\*Software Engineering Institute's views and beyond approach \[BBCG10\]\*](#) as well as [\*Phillipe Krutchen's 4+1 view model \[Kru95\]\*](#), [\*Rozanksi's and Woods's viewpoints and perspectives \[RW11\]\*](#) approach and Simon Brown's [\*C4 model \[Bro16\]\*](#).

We often organize viewpoints around quality attributes. Viewpoints can also be constructed to satisfy specific stakeholder needs. Here are a few examples:

- A *scalability, security, or maintainability viewpoint* will demonstrate how the architecture satisfies specific quality attribute scenarios.
- A *regulatory viewpoint* might provide a particular stakeholder group concerned with regulatory requirements information needed to perform an audit.
- A *teachability viewpoint* or *welcome to the team viewpoint* might walk a new teammate through your architecture and development practices with the goal of getting them to commit code on day one.
- A *business impact viewpoint* might show how different parts of the architecture contribute business value.

Viewpoints are a must in traditional architecture descriptions. With tribal and communal approaches, create viewpoints opportunistically and keep them lightweight. For example, once there are several ADRs in your code repository, create a viewpoint page that connects decision records together to provide overarching context. Mountains of documentation are not required to organize architecture descriptions for human consumption. Be kind to your readers and they will be delighted.

5. <http://www.iso-architecture.org/42010/afs/frameworks-table.html>



Views help us organize ideas so we can share the architecture effectively. There's more to an architecture description than just views and design decisions. Why you made the decisions in the first place is just as important.

## Explain the Rationale for Your Decisions

*Design rationale* describes why we made each design decision. Maybe you chose a pattern to promote a certain quality attribute. Maybe you picked a technology the team is familiar with or one that had the right cost. Every decision had alternatives, pros, and cons. We weighed trade-offs among properties and justified our conclusions through some logical thought process.

The better downstream designers understand the rationale for your decisions, the better they can embrace the *intent* of your design. The better others understand the intent behind your decisions, the more likely they will be to uphold the architectural integrity of the system as it evolves.

Rationale in an architecture description comes in many forms. It might be prose, a story, or a couple of bullet points. Sometimes your pile of rejected decisions can be more telling than a lengthy explanation.

## Describe the Paths Not Taken

Developing software is a journey. The path is winding and there are dozens of roads leading to the same destination. Every fork in the road, every decision you make, is an opportunity to help others understand why the software architecture is the way that it is. One method for helping people understand the decisions you made is to enumerate all the options you rejected.

Consider the paths not taken in the Project Lionheart architecture. See the [table on page 156](#).

Not everyone can be in the room when we make a decision. Reading the paths not taken lets you play back decisions so others can get a feeling for the architecture's journey. Without this knowledge, it's like watching the last five minutes of a movie. You missed all the character development and anything the characters do at the end will seem random. When you have the background, Darth Vader's decisions at the end of the *Return of the Jedi* hold a deeper meaning. Without the background, you might think Vader is just fickle and annoyed.

You will always reject many more ideas than you keep. Humans can dismiss dozens of options in the blink of an eye without even realizing it. Formally record any decision you discuss as a team. If you notice you're having an intense inner monologue, try to externalize your thoughts so others can benefit.

| Path not taken                                                         | Discussion                                                                                                                                                  |
|------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| One huge web application                                               | Doesn't allow us to schedule computationally intensive operations                                                                                           |
| Java for Display Business service                                      | Several teammates know Node.js, promote maintainability with JavaScript in the client and server                                                            |
| Index RFPs in SQL database, search service reads the database directly | Doesn't give an expressive enough query syntax required by stakeholders                                                                                     |
| MongoDB for data storage                                               | While it seems like a good fit, no expertise on the team                                                                                                    |
| Isolate services to their own containers                               | Not enough familiarity with technology. We can ship sooner putting everything on the same VM. This decision is not binding. We can change strategies later. |



### Get Your Hands Dirty: Describe the Paths Not Taken for a Project

Think about a recent project. Write down some of the alternative architecture choices that your team considered and why you rejected each option. Of these decisions, who on your team can describe the rationale for the decision?

Here are some things to think about:

- Were there any major discussions you had as a team?
- What decisions did your team struggle to make?
- What decisions were made under uncertainty?
- Were there any *point of no return* decisions that forced you down a particular path?

## Project Lionheart: The Story So Far...

The team has done a good job of making diagrams and sketching during design discussions, but pictures of whiteboards are not a replacement for a good old-fashioned architecture description.

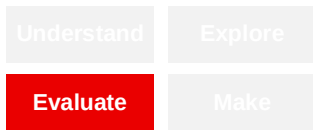
Since our team is small and co-located, you decide to continue using whiteboard sketches and system metaphors, but you encourage us to also start recording design decisions as architecture decision records. Metaphors and whiteboard drawings let us thrash quickly and cheaply. Recording ADRs as we make decisions creates a public record that is easier to share among the team and with future developers.

You decide to delay creating a formal architecture description. A more official document should be easier to write later in the life cycle. The ideas it contains will also be less likely to expire before we finish preparing the document since most design decisions will be set in code by then. Our primary audience is the next team after us. When the time comes, you think it will make sense to create teachability, deployment, and strategic change (extensibility) viewpoints.

## Next Up

Architecture descriptions can be great. It's up to you to design them for awesomeness. Think about what your audience needs to know and organize views of the architecture to help people see the whole picture. Describe the elements, their responsibilities, and why you chose them for this architecture. Take advantage of the breadth of description methods. Stuffy, big documents are not the only way to explain design decisions. The most important thing is to communicate effectively and explain your vision for the architecture to the world.

Creating an architecture description also creates our first opportunity to test the system's design, even before we've written much code. The ability to test the architecture early is great news since, as you'll see next, the earlier you can evaluate the architecture, the less pain you'll have down the road.



## CHAPTER 12

# Give the Architecture a Report Card

School report cards are an important feedback loop for students, parents, and teachers. Instead of waiting until the end of the year to learn if you pass or fail a subject, a quarterly report card can tell you if you're on track to get the grades you want and show you where you can improve before it's too late. Taking the time to evaluate our architecture, like school report cards, can help us catch problems early so we can stay on track to deliver.

Instead of thinking of evaluations as taking away time from programming, think of them as a way to make the time we spend programming even more powerful. Evaluations can take as little as an hour, and we can even fold them into existing development processes without anyone being the wiser.

In this chapter, you'll learn how to give the architecture a report card. The feedback from an evaluation can be used to educate the team, create buy-in for design decisions, reduce delivery risks, and improve the architecture.

## Evaluate to Learn

*Architecture evaluation* is a process by which we learn the extent to which an architecture is fit for purpose. A common fallacy when designing software architectures the belief that we should check the architecture only once, at the end of a design phase. If the whole architecture isn't correct, then everything fails and the implementation can't start. This thinking is wrong, wrong, wrong.

Architectures are never wholly good or wholly bad. Just like we can't see the entire architecture in a single view, we also can't evaluate the whole architecture in a single go. It's possible for a single component to be well designed, or one area of the architecture to be well understood while others are filled with risk. Not everything is required to be fully baked to begin implementation.

One way to use architecture evaluations is as a ceremonial sign-off. In cases where components must strictly meet specific criteria, such sign-offs can be extremely valuable. For example, a sign-off evaluation can avoid costly rework for systems with many integration points or significant hardware needs.

While sign-offs are sometimes important, using evaluations only as a phase gate check is a missed opportunity. The world constantly changes around us. The only way to know whether our architecture still satisfies our stakeholders' needs is to evaluate it.

During our evaluations we want to learn two things: *How good is the architecture?* and *How is the architecture good?* To answer these questions, we'll use what we know about the architecturally significant requirements (ASRs). The better the architecture satisfies the ASRs, the better the architecture fits its purpose.

## Test the Design

Test early, test often. This idea is as true for code as it is for architecture. Even when the architecture is only in its infancy, before we've written any code, there's always something we can test.

There are three things we'll need to evaluate an architecture. First, we need an artifact, a tangible representation of the design. Next, we need a rubric, a definition for *better* or *worse* from the stakeholders' perspective. Finally, we need a plan for helping reviewers generate insights so they can form an opinion about the goodness of the architecture.

## Make Something to Evaluate

Before we can evaluate anything, we need to have something to evaluate. We can evaluate real things, not ideas. We need to prepare a tangible artifact to evaluate. The artifact could be as simple as a whiteboard sketch or as detailed as a full architecture description.

Tangible things to evaluate are easy to find. Here are some ideas:

- Write some code.
- Sketch a model on paper or a whiteboard.
- Draw a model in a diagramming application.
- Prepare a slide-based presentation with different views of the architecture
- Summarize the results of an experiment in a presentation or whitepaper.



**Ipek says:**

## **Lines Are Also First-Class Citizens!**

*By Ipek Ozkaya, senior member of technical staff at the Software Engineering Institute at Carnegie Mellon University*

My work involves helping organizations and teams improve their systems' quality from the perspective of the fitness of its architecture. An unavoidable and obvious request in these engagements is "Show me your architecture." Over the years, based on the responses I get, I've developed a personal catalog of misconceptions about architecture and architecting.

**A printout of all of the sequence diagrams for all the use case scenarios you thought of so far is *not* your architecture!** While the collection of all your use cases and their behavioral traces, such as those you capture in sequence diagrams, are useful and important to your system, they do not provide the right level of abstraction to reason about classes of behavior of the system.

**Code review can *not* replace an architecture review!** The bottom line of any architecting effort is to design and implement a system that meets its business and stakeholder goals. Working code is the inevitable reality. However, architectural concerns cross-cut many elements in the implemented system. An effective architecture review is bound by those architecturally significant requirements and all the elements they touch. Traditional code review practices do not cover this end-to-end perspective.

**Boxes are *not* the only architectural elements!** In cases when a team does have an artifact, often called the Software Architecture Document, and sadly referred to as SAD, the document contain depictions of ad hoc box-and-line drawings. This is a great start, but discussions are long on the boxes while the lines are completely forgotten. This is unfortunate because many times the lines carry the most critical aspects of the architectural decisions.

Of these three misconceptions, the most significant one is to appreciate the importance of lines in software architecture diagrams. Think about it. If you aim to increase performance, then you focus on the frequency and volume of the inter-element communications. If you want to increase modifiability, you limit interactions between elements. If you want to optimize security, you protect the inter-element relationships. All of these are represented by the lines!

Many architectural decisions are carried on those thin, often forgotten lines. Since so many architectural decisions are carried by the inter-element relationships, one of my first recommendations to teams is to get a better understanding of the relationships between the elements. Treat the lines as first-class citizens in your architecting journey!

- Create a traditional architecture description ([introduced on page 147](#)), architecture haiku (see [Activity 21, Architecture Haiku, on page 263](#)), or set of Architecture Decision Records ([Activity 20, Architecture Decision Records, on page 260](#)).
- Build a utility tree showing which quality attributes are promoted by different components.

In [The Four Principles of Design Thinking, on page 15](#) you learned to make things tangible to facilitate communication. The artifact used during an evaluation will communicate our best intentions for how we plan to (or in some cases, already have) addressed the ASRs.

Prepare artifacts that are likely to solicit the type of feedback we want. For example, if we want reviewers to focus on a specific quality attribute, then the artifacts should include views relevant to that quality attribute. If the architecture is young and we want general feedback, consider using *sketchier* diagrams to show that the design is in flux. If the design decision is about something high risk and high cost, favor greater precision and formality to communicate the seriousness of the matter at hand.

## Define a Design Rubric

Every architecture exhibits shades of *right* and *wrong*. One reviewer may conclude the architecture is a masterpiece whereas another proclaims it a dumpster fire. A *design rubric* defines the criteria reviewers should use when judging the fitness of the architecture.

Rubrics consist of two parts. *Criteria* describe the characteristics used to evaluate the design artifacts. *Ratings* describe the scale used to interpret the characteristics. Typically, rubrics take the shape of a matrix.

In the [example on page 163](#) from Project Lionheart we're using quality attribute scenarios as the criteria, listed on the left. On the right, reviewers enter their ratings using the provided scale, which is described at the bottom.

Let's explore how we arrived at this rubric and define some general advice for creating rubrics.

### Select Criteria Based on Architecturally Significant Requirements

Architecturally significant requirements define the software's purpose from a stakeholder's perspective. In [Chapter 5, Dig for Architecturally, on page 49](#), you learned how to specify ASRs in a way that enables analysis and evaluation. If we define ASRs in a precise, unambiguous, and measurable way, then we can use them to help define a rubric for evaluation.



| Quality Attribute | Scenario                                                                    | Rating (1 - 4) |
|-------------------|-----------------------------------------------------------------------------|----------------|
| Availability      | Lionheart responds even when the index is unavailable.                      |                |
| Availability      | Results are always returned except during maintenance.                      |                |
| Performance       | Results are visible within 5 seconds under average load.                    |                |
| Scalability       | The system can expand to handle 5% annual data growth for the next 7 years. |                |

**Rating Scale**

- 1 = Does not meet expectations for scenario or unable to evaluate
- 2 = Partially satisfies scenario or satisfies the scenario but with unacceptably high risk, technical debt, or cost
- 3 = Satisfies scenario with acceptable risk, technical debt, and costs
- 4 = Satisfies scenario with little or no risk or technical debt and within budgets


 Defines how to score the criteria

Using the ASRs as a guide, we can select a rubric's criteria. The best rubrics meet the following conditions:<sup>1</sup>

**Important and essential** The criteria in a rubric defines what we think a good architecture should look like relative to the ASRs. Criteria should not include ideas that are *nice to have* or frivolous details not required for the architecture to be fit for purpose.

**Distinct** Criteria within the same rubric should not overlap with one another. Each criterion is one facet of the overall fitness of the design. Ideally each criterion can be assessed and scored independently.

**Observable and measurable** Reviewers must be able to assess and score the criteria in the rubric. The artifacts we prepare for the evaluation will make criteria visible. The activities we perform during the assessment will collect data that lets us measure the criteria.

**Precise and unambiguous** Every reviewer should interpret the criteria in the same way.

Quality attribute scenarios should already meet these recommendations and always make for good criteria.

1. [http://www.ucdenver.edu/faculty\\_staff/faculty/center-for-faculty-development/Documents/Tutorials/Rubrics/index.htm](http://www.ucdenver.edu/faculty_staff/faculty/center-for-faculty-development/Documents/Tutorials/Rubrics/index.htm)



Decide the Rating Scale for Criteria

During the evaluation reviewers will judge the criteria using a provided rating. Rating scales define what *needs improvement*, *good*, *better*, and *best* looks like. The size of the scale depends on the goals of the evaluation. Here are some different rating scales and when they might be appropriate to use:

| Scale Size | Examples                                                                      | Use it when...                                                                                                           |
|------------|-------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| 2          | yes or no; condition satisfied or not satisfied                               | Acceptance is all or nothing for a condition, standard, or presence of an item; single or few reviewers                  |
| 3          | fail, pass, or awesome; never, sometimes, or always; low, acceptable, or high | There is a minimum acceptable threshold but also a preferred expectation for the design; multiple reviewers are involved |
| 4          | never, sometimes, usually, or always; fail, fair, pass, or exceed             | Detailed feedback is desired; expectations can be <i>nuanced</i> or involve multiple pieces                              |
| 5+         | choose a number 1–10                                                          | Avoid using. Too many options in the rating scale lead to inconsistent reviews.                                          |

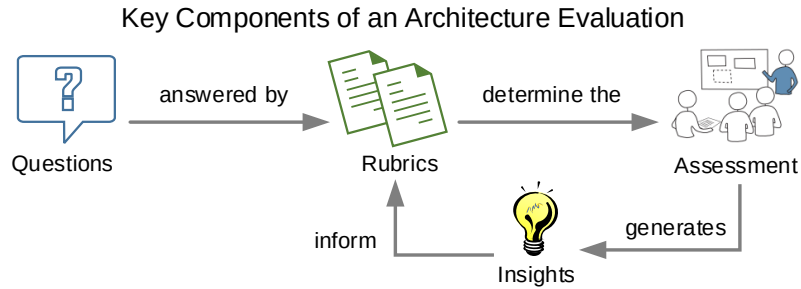
In our example from Project Lionheart [on page 162](#), we chose to use a simple 1–4 scale so reviewers could offer more feedback about the design. To use the rubric, multiple reviewers will score the criteria and we’ll average the results. We’ll also highlight any criteria that scored a 1 for further discussion, even if the criteria have an acceptable average score.

This example rubric captures scores well but doesn’t have space where reviewers can explain *why* they scored criteria the way they did. Scores are an easy way to assess the design quickly, but knowing what reviewers were thinking when they scored different criteria is invaluable information.

We have artifacts. We have a definition for *better* and *worse*. The last step is to help reviewers score the rubric by helping them generate insights about the architecture.

Generate Insights

Design rubrics contain answerable questions as shown in the [figure on page 165](#). We find the answers by helping reviewers generate insights about the



design, which reviewers use to form opinions about how well the design satisfies the ASRs.

We can generate insights in a number of different ways, such as questionnaires, directed explorations, risk elicitations, or code analysis. To help decide which activities will bear the most fruit, we'll need to figure out what information is required to answer our rubric. Here are a few examples:

| Rubric Criteria       | Insights to help score the criteria                                                                                                                                                            |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Amount of Risk        | Identify risks with risk storming ( <a href="#">described on page 301</a> ) or a general risk elicitation workshop; examine the number and severity of risks identified                        |
| Amount of Uncertainty | Generate open questions with a question-comment-concern workshop ( <a href="#">described on page 298</a> ); examine the number of open questions and estimate how difficult they are to answer |
| Reviewer Consensus    | Use multi-voting, surveys, thumbs up/down, and ratings                                                                                                                                         |
| Design Completeness   | List known components and their current design state; define a threshold for <i>complete</i> and <i>more work needed</i>                                                                       |
| Fit for Problem       | Walk through quality attribute scenarios ( <a href="#">described on page 307</a> ) and identify sensitivity points, problem areas, risks, and questions                                        |
| Technical Debt        | List value adding use cases that cannot be implemented with the current architecture; estimate the cost to prepare the architecture for the use case                                           |
| Quality               | Count the number of defects by architectural component and define a threshold for <i>high</i> and <i>low</i> quality                                                                           |

The bulk of the effort in an architecture evaluation is spent generating insights. Since insights are often generated collaboratively during workshops, let's learn more about how to plan and facilitate an architecture evaluation workshop.



### Get Your Hands Dirty: Ask Seven Good Questions

Good evaluators ask the right questions. Learning to ask the right questions takes practice. Write down seven or more questions about the architecture of a recent project for which you don't know the answer. Why seven? We want to move past the obvious to find interesting things others might have missed.

Here are some things to think about:

- Be specific. General questions only provide general insights. The more specific the questions, the more actionable your insights.
- What do you know (or not know) about the relations in the architecture?
- Are there one or more views of module, component and connector, and allocation structures?
- What worries you? Playing *what if...* is not a fun game, but worries are often the seed of real engineering risks.

## Host an Evaluation Workshop

The goal of an *architecture evaluation workshop* is to gather and analyze the data necessary to assess the architecture. By the end of the workshop, we should be in a position to qualify how well the architecture satisfies desirable quality attributes and other ASRs.

While there are many ways to run an evaluation workshop, all workshops follow the same basic formula:

1. *Prepare*—Find or create required artifacts. Define rubrics. Select methods to gather data and invite reviewers.
2. *Prime the Reviewers*—Share the artifacts and rubrics with reviewers. Explain the goal of the evaluation and answer reviewers' questions. Reviewers should fully understand the artifacts, rubrics, and purpose of the evaluation before starting the assessment.
3. *Assessment*—Lead reviewers through activities to explore the artifacts and generate insights needed to score the rubric. Activities can be collaborative or solo.

- 4. *Analyze*—Compile the data provided by reviewers. Summarize the results and look for trends.
- 5. *Follow-up*—Decide on next steps based on what you learned during the workshop.

Let’s take a closer look at each of these steps.

Prepare for the Evaluation

As a part of our preparation, we must decide the goals of the evaluation and develop whatever artifacts we need to meet those goals. Here are some examples:

| If you want to evaluate...                           | You might need artifacts like...                                                                                               |
|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| How well a specific quality attribute is promoted    | Views relevant to the quality attributes of interest, test results, use cases, quality attribute scenarios                     |
| Technology or pattern options                        | Technology or pattern descriptions, experiment overviews, experiment results, quality attribute scenarios                      |
| Likelihood of hitting cost or schedule targets       | Component overview, component estimates, technical dependencies, team capacity                                                 |
| Design evolution path                                | Overviews of the current and to-be architecture, list of evolution steps                                                       |
| Architecture description completeness or correctness | An architecture description, questions that should be answerable by looking at the description, description quality checklists |
| Security                                             | Abuse cases, misuse cases, threat models, data stores, views needed to identify sensitivity points and attack vectors          |
| Release Readiness                                    | Quality attribute scenarios, relevant views, release checklists, test results                                                  |

In addition to preparing artifacts, we’ll need to create rubrics and decide what data is required to score them. If you plan to run the assessment as a workshop, then prepare the agenda and any materials needed to host the workshop.

When selecting reviewers, look for stakeholders and non-stakeholder experts who are detail-oriented and care about the system being designed. Ideal candidates will have relevant domain knowledge or expertise in the technologies and patterns used in the architecture. They will also be prepared to offer

an objective assessment. As few as two reviewers can perform an assessment, but it's possible to involve dozens of reviewers if required.

After the reviewers have committed to participate in the evaluation, we'll need to prepare them to do a good review.

## Prime the Reviewers

All reviewers should have the information needed to provide good feedback. Present necessary background information to reviewers, such as the system's context, architecturally significant requirements, and the artifacts under review. Answer any questions the reviewers have about the context, rubrics, and goals.

A slide deck or whiteboard talk works well for this. When conducting an evaluation within your team, consider creating artifacts and reviewing context together, just-in-time at the start of the workshop.

Once the reviewers are primed and ready, it's time to perform the assessment and generate some insights.

## Facilitate the Assessment

During the assessment part of the workshop, we'll generate insights by guiding reviewers through a series of activities designed to illuminate inconsistencies in thinking or highlight potential problem areas. Many activities can be used to generate insights. Here are a few examples, which are described fully in [Chapter 17, \*Activities to Evaluate Design Options\*, on page 285](#):

- Many evaluations will use some form of a scenario walkthrough as [described on page 307](#). This activity is the most basic and reliable architecture evaluation tool.
- The Question–Comment–Concern activity [described on page 298](#) is a form of visual brainstorming that helps reviewers quickly surface facts and questions about the architecture.
- Risk storming, [described on page 301](#), is also a form of visual brainstorming, which focuses exclusively on risks in specific views of the system.
- If the goal of the evaluation is to compare and contrast alternatives, we might use the Sketch and Compare activity as [described on page 311](#) to pit two or more ways of promoting the same quality attributes against one another.

- Code review, [described on page 289](#), is not reliable on its own for finding architectural problems but it can identify misalignment between detailed design and the architecture. Code review is also an excellent tool for keeping tabs on static structures as they emerge.
- If we've recorded ADRs for our system (see [Activity 20, Architecture Decision Records, on page 260](#)), then we can replay the design decisions and decide whether those decisions still hold true. Proposed ADRs can be evaluated for fit as well.

Choose evaluation activities based on what we need to learn, the time available, and the stakeholders' familiarity with architecture evaluations. A small, experienced evaluation team of only 3–4 people can generate interesting insights with a simple question–comment–concern activity in as little as 60 minutes. A less experienced group might yield better results by walking through scenarios or design decisions explicitly.

During an evaluation workshop, we want to determine whether the architecture passes our criteria, but this shouldn't be the only outcome from the workshop. We also want to learn how to improve the architecture's design, not just that it needs improvement.

## Analyze Data and Reach Conclusions

No matter what criteria we use during the evaluation, we want a clear and definitive conclusion. Explicitly state how well the architecture stood up to the criteria used to evaluate it and make concrete recommendations for how the architecture can be improved. The conclusions from an architecture evaluation should not be a simple pass or fail.

Whether the architecture is fit for purpose is only half the story. It's just as important to understand why the design is fit (or not) for purpose. Great designs can always be improved. Even a poor design will get some things right.

Use the insights generated during the evaluation to look for trends and opportunities. To decide *how* the architecture is good, look for risks and open questions. Risks show where the design might allow bad things to happen relative to criteria assessed in the workshop. Open questions shine a light on gaps in communication or knowledge about the architecture.